

From weighted independent set to an exact pseudoforest DP: a derivation, and a comparison with Tree-DP

Abstract

We derive an exact algorithm for the “one differential candidate per shape” restriction of the gapped-CSA compression problem, starting from the general weighted-independent-set formulation of exact compression and showing step by step why the general problem resists tractable structure, why restricting to *preferred* candidates removes that obstruction, and why the resulting conflict graph is always a *pseudoforest* (treewidth ≤ 2) rather than merely a forest. We give the resulting linear-time DP and compare it, on the existing running example, against the **Tree-DP** algorithm already implemented in `compress.hpp` and documented in `dep_order_walkthrough`. The two coincide on that example; we explain precisely why, and identify the structural situation in which they would diverge.

1 The exact objective

Fix the gapped suffix array (GSA) of the LexText, with m ranks grouped into LCP-intervals $I_c = [\ell_c, r_c)$ by first symbol (shape) c . A **candidate** is a triple $k = (T_k, S_k, a_k)$: a contiguous target run $T_k \subseteq I_c$ for some shape c , a contiguous source run S_k disjoint from I_c , and a shift $a_k \in \{1, \dots, a_{\max}\}$, such that shifting every rank of S_k forward by a_k symbols reproduces T_k bijectively and in order (the alignment condition of the walkthrough, §1.3). Write $\text{cov}(k) = |T_k|$.

Compressing means choosing a set of candidates $X \subseteq \mathcal{C}$ to *accept*. A rank is dropped from the stored array C iff it lies in T_k for some accepted k ; $|C| = m - \sum_{k \in X} \text{cov}(k)$. Two constraints make X valid:

- **Target-disjointness**: every rank lies in T_k for at most one accepted k (a rank can be recovered from only one place).
- **Source-integrity**: if $k \in X$, no rank of S_k may lie in $T_{k'}$ for any $k' \in X$ (a source must be physically stored, not itself virtual — this is exactly what `run_kept_` checks before an accept).

So exact compression is

$$\max_{X \subseteq \mathcal{C} \text{ independent}} \sum_{k \in X} \text{cov}(k)$$

in the conflict graph $G = (\mathcal{C}, E)$ where $\{k, k'\} \in E$ iff $T_k \cap T_{k'} \neq \emptyset$ or $S_k \cap T_{k'} \neq \emptyset$ or $S_{k'} \cap T_k \neq \emptyset$. This is weighted maximum independent set (equivalently weighted set packing) on G .

2 Why the general problem resists tractable structure

2.1 Generic hardness

Weighted independent set is NP-hard on general graphs, and even the decision version (“is there an independent set of size $\geq \kappa$?”), parameterized by κ is W[1]-complete — so no $f(\kappa) \cdot n^{O(1)}$ algorithm is expected regardless of how the parameter is chosen. This says nothing by itself about *our* G , which is far from arbitrary; the question is whether G ’s specific structure escapes this.

2.2 Per-shape cliques are not bounded by $a_{\max}$

Under the storage format actually implemented (`HashEntry` holds one H_{offset}), at most one candidate may be accepted per shape, so all candidates targeting the same I_c mutually conflict — a clique. A treewidth lower bound is (clique size -1), and the clique size for one shape is not simply $a_{\max}$: for a fixed a , `enumerate_candidates_` can still emit several disjoint maximal runs, so a shape’s total candidate count can scale with $a_{\max} \times |I_c|$ in the worst case. The single-slot storage restriction is therefore already enough to make the raw conflict graph impractical to decompose.

2.3 Laminarity of ranges does not make G a tree

Every S_k and T_k is (or extends to) an LCP-interval: S_k is built from a maximal run with $\text{LCP} \geq a_k + 1$, and because prepending symbols preserves relative order, its image T_k shares that same extended common prefix, hence is itself an LCP-interval. LCP-intervals are exactly the internal nodes of the suffix tree, and any two of them are either disjoint or nested — never partially overlapping (a standard suffix-tree fact: sharing a longer prefix is a strictly stronger, all-or-nothing condition than sharing a shorter one, so if two prefix-classes intersect at all, the deeper one is wholly contained in the shallower one). One might hope this makes G itself tree-shaped. It does not, and it is worth seeing exactly why, since the gap is instructive.

A candidate is not a single point in this hierarchy — S_k and T_k are *disjoint*, hence incomparable, so a candidate is a cross-link between two unrelated branches of the tree. Laminarity constrains any two intervals that intersect *each other*; it places no constraint across three or more independently-chosen cross-links. Concretely, nothing prevents three candidates k_1, k_2, k_3 in unrelated parts of the tree with $S_{k_1} \subseteq T_{k_2}$, $S_{k_2} \subseteq T_{k_3}$, $S_{k_3} \subseteq T_{k_1}$ — three independent containments chained into a genuine 3-cycle in G , with no pairwise relationship required between, say, T_{k_1} and T_{k_2} themselves. So the raw candidate conflict graph can contain cycles that have nothing to do with the interval hierarchy being non-tree-shaped; laminarity of the *ranges* does not transfer to tree-shapedness of the *conflict graph over candidates*.

3 Restricting to one candidate per shape

3.1 Definition

For each shape c with a candidate of coverage ≥ 2 , keep only its **preferred** candidate — maximum coverage, ties broken by larger a (exactly the notion already used to build the DepOrder preference DAG and the Tree-DP forest, §2 of the walkthrough), with *no further threshold*. This yields at most one candidate node per shape.

3.2 The clique disappears

Since each shape now has a single candidate (or none), there is nothing left to form a clique out of: the decision per shape is binary, “accept my one preferred candidate or not,” contributing a domain of size 2 rather than a clique of unbounded size. This is the standard fix for a “choose-one-of-many” clique: model it as one variable with a domain, not as many mutually-exclusive nodes.

Fact 1 (Out-degree ≤ 1). *A candidate’s source lies entirely inside exactly one other shape’s interval.*

Proof. S_k has $\text{LCP} \geq a_k + 1 \geq 2$ throughout, so every rank of S_k shares its own first symbol (position 0) with every other rank of S_k — i.e. all of S_k lies in a single I_w . $\square$

So, viewing “shape c conflicts with shape w ” as a directed edge $c \rightarrow w$ whenever c ’s preferred candidate’s source lies in I_w , every node has out-degree ≤ 1 . As an undirected graph this is a **pseudoforest**: each connected component has at most as many edges as nodes, hence contains at most one cycle (a *unicyclic* component), with trees hanging off it. Pseudoforests have treewidth ≤ 2 .

3.3 Rank-awareness only sparsifies further

An edge $c \rightarrow w$ in the preference graph is necessary but not sufficient for a genuine conflict: c ’s own preferred candidate covers a specific sub-run of I_c ’s own ranks, and w ’s preferred candidate (if $w = c$ ’s source) needs a specific sub-run of I_w ’s ranks; these only actually collide if the required ranks overlap (the “GC **SKIP**” step of the walkthrough’s Phase I is exactly an instance where they did collide). Checking real overlap can only *delete* edges from the preference graph, never add new ones — a shape can never conflict with another shape it had no preference relationship with in the first place. So the true, rank-aware conflict graph is a subgraph of the preference graph, hence still a pseudoforest. The rank check makes the instance easier, never harder.

4 An exact linear-time algorithm

Because the true conflict graph is a pseudoforest, weighted maximum independent set is solvable exactly, in linear time, with no appeal to generic treewidth machinery.

Tree parts. For a node v with children $\text{ch}(v)$ (shapes whose preferred source is v), and a boolean src_ok recording whether v ’s own preferred source is currently available to it, define

$$\begin{aligned} \text{DP}(v, \text{true}) &= \min \left(\underbrace{\text{isize}(v) + \sum_{w \in \text{ch}(v)} \text{DP}(w, \text{true})}_{\text{KEEP}}, \underbrace{(\text{isize}(v) - \text{cov}(v)) + \sum_{w \in \text{ch}(v)} \text{DP}(w, \text{false})}_{\text{COMPRESS}} \right), \\ \text{DP}(v, \text{false}) &= \text{isize}(v) + \sum_{w \in \text{ch}(v)} \text{DP}(w, \text{true}) \quad (\text{COMPRESS forbidden: forced KEEP}). \end{aligned}$$

This is exactly the KEEP/COMPRESS recurrence already implemented for Tree-DP (§*KEEP versus COMPRESS* of the walkthrough) — nothing new is needed for the tree portions of the pseudoforest.

Cyclic parts. A component with one cycle $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_L \rightarrow v_1$ (each v_i ’s preferred source is v_{i+1} , indices mod L) plus trees hanging off each v_i cannot be handled by the tree recurrence alone, because “is v_i ’s source available” depends circularly on whether v_{i+1} is kept, which depends on whether v_{i+2} is kept, $\dots$, back to v_i . This is solved by the standard fold-the-cycle trick: fix the decision for v_1 (KEEP or COMPRESS), which breaks the circular dependency and turns the rest of the cycle into a path; run the tree/path recurrence under that fixed assumption; take the better of the two fixings. This is $O(L)$ per cyclic component (each is small in practice — a mutual pair, as with CT/GC, is the typical case) and exact: no candidate information is discarded, unlike dropping an edge to force acyclicity.

Complexity. Every shape appears in exactly one component; tree parts are $O(1)$ amortized work per node, cyclic parts are $O(L)$ per component with $\sum L = O(\#\text{shapes})$. The whole pass is $O(N)$, matching Tree-DP’s own complexity, with strictly more of the instance solved exactly (no filtering, no arbitrary edge-dropping) and no ad hoc thresholds.

5 Worked check: does this match the running example?

On $T = (\text{GCCTTTAAAG})^3$, $S = \#.\#$, the preference graph (no $\text{cov} \geq 3$ filter, no $|I_c| > 2$ filter) is exactly the mutual pair $\text{CT} \leftrightarrow \text{GC}$ with leaves AG, TA hanging off CT and AA, TT hanging off GC — a single unicyclic component ($L = 2$) with four tree leaves attached. Folding the 2-cycle by fixing CT ’s decision:

fix	feasible?	total kept (CT,GC,leaves)
CT KEEP, GC KEEP	yes	$6 + 5 + 0 + 0 + 0 + 0 = 11$
CT COMPRESS (via GC), GC KEEP	yes, but AG/TA lose CT as a source	$4 + 5 + 5 + 6 + 0 + 0 = 25$
CT KEEP, GC COMPRESS (via CT)	yes, but AA/TT lose GC as a source	$6 + 3 + 0 + 0 + 3 + 3 = 15$
CT COMPRESS, GC COMPRESS	infeasible	each needs the other kept as its source

The minimum is 11 (both kept), giving $|C| = 11 + 3 \text{ singletons} = 14$ — identical to both `DepOrder` and `Tree-DP`. This is not a coincidence special to the algorithm; it is because, on this instance, keeping both hubs genuinely is the cost-minimizing choice by the numbers, so every method that correctly evaluates “keep both” as an option lands on it.

6 Comparison with Tree-DP

	Tree-DP (implemented)	Pseudoforest DP (this note)
Candidate universe	preferred candidate per shape, restricted to $ I_c > 2$ and forest edges with $\text{cov} \geq 3$	preferred candidate per shape, every shape with $\text{cov} \geq 2$ (the base validity threshold used everywhere else)
Graph shape	<i>forced</i> acyclic: an edge is dropped outright (<code>reaches()</code> check) whenever adding it would close a cycle, in <code>unordered_map</code> iteration order	pseudoforest <i>as it naturally occurs</i> : cyclic components are solved exactly by folding, never by discarding an edge
Handling of a genuine mutual/cyclic preference	one direction is silently dropped; which one depends on hash iteration order, not on which choice is better	both directions are weighed against each other explicitly (the four-row table above), and the correct one is kept
Special-casing	roots need an extra check (“is the preferred source outside my own subtree”) to patch exactly the case where a self-referential edge was dropped	no special case: the cycle-folding step handles this uniformly for every node, root or not
Nodes with only a weak ($\text{cov} = 2$) or small ($ I_c \leq 2$) preference	never enter the DP; resolved later by the size-ordered leftover greedy pass	enter the DP like any other node; resolved exactly, subject to the same pseudoforest structure
Complexity	$O(N)$	$O(N)$

6.1 When do they actually diverge?

They provably compute the *same* answer whenever the preference graph, restricted to $\text{cov} \geq 3$ edges among $|I_c| > 2$ shapes, happens to already be acyclic — which is exactly the case on the running example, since the only cyclic pair (CT, GC) has coverage 2 on *both* directions and is filtered out by `Tree-DP`’s own threshold before the cycle-avoidance logic is even exercised. The two methods have not yet been distinguished by this instance; they happen to agree because the instance’s one cycle is weak enough to be invisible to `Tree-DP`’s filter.

They would diverge on an instance where a genuine $\text{cov} \geq 3$ cycle exists among the preferred candidates — e.g. three shapes A, B, C each preferring the next ($A \rightarrow B \rightarrow C \rightarrow A$, all coverage ≥ 3) with different, possibly asymmetric leaf coverage hanging off each. Tree-DP would keep two of the three edges (whichever survive `reaches()` in whatever order `prefs` happens to iterate) and force the third node to KEEP outright without ever weighing it against the alternative, whereas the cycle-folding DP evaluates all L ways of breaking a length- L cycle and provably picks the best one. Tree-DP’s answer in that case would be a valid but not necessarily optimal point in the same search space; whether it actually loses anything depends on which edge iteration order happens to drop, which is incidental to the algorithm rather than a designed choice.

7 Scope and what remains heuristic

Both methods are exact only for the **restricted** search space of “one candidate per shape, and only its most-preferred one.” Real compression sometimes benefits from a shape accepting a *non*-preferred (lower individual coverage) candidate because it is more compatible with its neighbors — this is precisely what the shared Phase II unpin/retarget pass chases heuristically, in both Tree-DP and DepOrder, and is not addressed by either DP. Extending exactness to that larger candidate universe reintroduces the obstructions of §3 (per-shape cliques of unbounded size, and conflict cycles unrelated to the interval hierarchy) and remains open. What this note establishes is that the *first-order* approximation — one candidate per shape — was already exactly solvable all along, in the same asymptotic time Tree-DP spends approximating it, simply by folding rather than discarding the one place where the preference graph can loop back on itself.